The Algebra of a Vote

One recursive structure for citizens, parties and sub-parties;
its algebra; and the exact gap where cycles live

Claude, 14 September 2026

Design only; nothing is built. This supersedes a draft called *Two Trees*, which had a delegation tree for citizens and a separate amalgamation tree for parties. There is one tree. A party is a citizen that happens to fan out somewhere, and it delegates to sub-parties and to individuals in exactly the way a citizen delegates to parties and individuals.

Collapsing the two made the mathematics better rather than merely shorter, and §3 is the reason this document exists.

The brief is in `PROBLEM.md`. In one line: every citizen votes on every issue, nobody has time, so a vote is a *delegation* — refinable issue by issue, all the way down — and the finest issues are settled where a delegate's vote counts once per citizen who sent it to them.

1 One container

$$\text{IssueC} \;=\; \text{Issue} \triangleleft \text{Vote}$$

A shape is an issue; a position is a valid vote *on that issue*. The dependency is not decorative: a referendum admits aye or nay, a seat admits a candidate from a list, a budget line admits a number in a range, and a vote of the wrong kind is not dissent but a type error.

A voter is a `Runner<IssueC>` — somebody an issue can be put to, who answers. That is the entire ontology, and it has no notion of citizen, party or independent in it.

2 One structure, recursively

Per issue, a voter does one of three things.

		backward map
`Leaf`	answers from its own head	—
`Relay` to one voter	$+$	the **identity**
`Fold` over many voters	$\boxtimes$	an **amalgamation**

The targets of `Relay` and `Fold` are voters, so they may be parties, sub-parties or individuals. Nothing distinguishes them.

```
type Voter =
  | { tag: "Leaf"; opinion: (i: IssueId) => Vote | null }
  | { tag: "Node"; assign: ReadonlyMap<IssueId, Edge> }   // inherited downward

type Edge =
  | { tag: "Relay"; to: VoterId }
  | { tag: "Fold";  over: readonly VoterId[]; rule: Rule; cache: Cache }
```

Assignments sit at nodes of the *issue* tree and are inherited downward until overridden, so a delegation is a partial function on that tree denoting a total function on its leaves. The lazy delegation — one party for everything — is the constant function, presented by a single assignment at the root.

A citizen:

```
(citizen)
|- health ................... Relay -> party C
|- education ................ Relay -> party A
|    '- art and creativity .. Relay -> Sarah Smith     <- overrides, this subtree
'- military ................. Relay -> party C
```

And party A, which is the same shape one level down:

```
(party A)
|- health ................... Relay -> Bea             <- her word is final
|- education
|    |- art and creativity .. Relay -> Ali
|    '- (everything else) ... Fold  -> Ali, Bea, Carl  <- majority
'- military ................. Fold  -> faction X, Carl <- a SUB-PARTY and a person
```

The old vocabulary survives as regions of one cube, not as types.

regular citizen	relays everywhere, never a leaf
politically active citizen	a leaf on the issues they care about, relays elsewhere
independent	a leaf, never folds
party	**folds somewhere**
party with a whip	folds, and caches the result (§6)
coalition	`Fold` with `Unanimous`

Observation. *Folding is available to anybody. A citizen may perfectly well say "on defence, poll parties A, B and C and take the majority." The type permits it with no special case, which is a good sign that the three cases are the right three.*

3 The algebra

The three cases are a container in the plainest sense:

$$F(X) \;=\; \underbrace{1}_{\texttt{Leaf}} \;+\; \underbrace{X}_{\texttt{Relay}} \;+\; \underbrace{\mathrm{List}(X)}_{\texttt{Fold}}$$

shapes $\{\texttt{Leaf}, \texttt{Relay}, \texttt{Fold}_n\}$ with positions 0, 1 and n.

An electorate is an F-**coalgebra** $V \to F(V)$: each voter maps to its behaviour over its sub-voters. Resolving a vote asks for an F-**algebra** on votes, $\alpha : F(\mathrm{Vote}) \to \mathrm{Vote}$, and its three components are already familiar:

summand	the map	which is
$1 \to \mathrm{Vote}$	your own opinion	a leaf
$\mathrm{Vote} \to \mathrm{Vote}$	the identity	**delegation's backward map**
$\mathrm{List}(\mathrm{Vote}) \to \mathrm{Vote}$	the fold	**a party's backward map**

Robin asked for "the algebra". It is an algebra, and its three components are exactly the three backward maps. Forward maps are routing and are easy; the content of the system is the algebra, which is what `federation.pdf` reports for cyclic federations from an unrelated direction — *the failure is entirely in the backward map.*

4 Where cycles live, exactly

Resolution is the unique coalgebra-to-algebra morphism, and it exists precisely when the delegation is well-founded.

Claim. *An acyclic electorate is an element of the initial algebra μF: a finite, well-founded tree. A cyclic one is an element only of the final coalgebra νF, because a cycle unfolds to an infinite tree.* ***The cycle problem is the gap between the two fixed points, and nothing else.***

This is `federation`'s finding restated for a new subject — containers type the interface and say nothing about the well-foundedness of the composite. `Voter` admits both fixed points. Only one of them terminates.

Practically, that licenses two things. Detection is `federation/mobius.ts`'s `Walk`, whose base case is *seeing itself in* `visited` and whose state rides in the prompt because a header cannot be a type. And termination needs `fuel` in the prompt as well, since nested parties make depth unbounded — so a fuel-exhausted branch is a *third* outcome beside cycling and abstaining.

A claim in `README.md` was wrong and is corrected here. It said each voter delegates to at most one representative, so the graph is *functional*, out-degree at most one, every component a tail into a single cycle.

A `Fold` has out-degree n. Resolution is a fold over a *tree*, not a walk down a path; cycle detection is per branch rather than per component; and the same voter may be reached by several routes, which is a question about weights before it is one about cycles.

Recursion makes this worse in a way worth stating plainly: a cycle may now run *between parties*, where no individual citizen did anything wrong. "Report it to the citizen so they can repair it" stops being an adequate policy at depth.

5 The types

```
type IssueId = string;                       // a path: "education/art"
type Kind =
  | { tag: "Binary" }
  | { tag: "Choice"; options: readonly string[] }
  | { tag: "Amount"; lo: number; hi: number };

interface Issue { id: IssueId; kind: Kind; children: readonly IssueId[] }

// The position depends on the issue's kind. This is what PosAt is for.
type VoteFor<K extends Kind> =
    K extends { tag: "Binary" } ? { tag: "Aye" } | { tag: "Nay" }
  : K extends { tag: "Choice" } ? { tag: "Pick"; option: string }
  : K extends { tag: "Amount" } ? { tag: "Amount"; value: number }
  : never;

type IssueC =
  | Fib<{ tag: "Poll"; issue: IssueId;
          visited: readonly VoterId[]; fuel: number }, Ballot>
```

```
  | Fib<{ tag: "Disclose"; issue: IssueId }, Disclosure>
  | Fib<{ tag: "Reset" }, { tag: "ResetAck" }>;

// A ballot ADMITS FAILURE. All three failures are positions, not exceptions,
// so every caller is made to say what it does about each.
type Ballot =
  | { tag: "Cast";      vote: Vote; weight: number }
  | { tag: "Cycle";     ring: readonly VoterId[] }
  | { tag: "Exhausted"; at: VoterId }
  | { tag: "Abstain";   who: VoterId; why: string };

// Transparency, and note that EVERY voter answers it. A leaf discloses "leaf".
// Uniformity of interface is load-bearing -- see the last section.
type Disclosure =
  { edge: "leaf" | "relay" | "fold"; rule?: Rule; cached: boolean };

type Rule =
  | { tag: "Majority" }    // one sub-voter, one vote
  | { tag: "Weighted" }    // by weight each personally carries -- see section 7
  | { tag: "Unanimous" }   // or abstain
  | { tag: "Plurality" };

type Cache =
  | { tag: "Live" }                                  // poll when asked
  | { tag: "Cached"; until: Term; line: Vote | null }; // decide once, then answer
```

6 The party line is a worker's state

Whether a fold is recomputed or remembered is **orthogonal** to which rule it uses, and the earlier draft got this wrong by listing `Line` as a fourth rule beside `Majority` and `Unanimous`.

A *live* party polls its members when asked. A *whipped* party polls them once, writes the result into its state, and afterwards answers from the state. That is a backward map writing and a forward map reading — the same construction as the CEO in `firm`, for the same reason: one may not branch on a reply, so one writes the reply into the state and branches next tick.

Observation. *The whip is the one-tick lag. A whipped party answers today according to what it decided yesterday, which is what a whip is for and what it costs.*

The political difference is real and is why `Disclose` exists:

> Delegating to a **live** party is delegating to its current membership. Delegating to a **whipped** party is delegating to a past decision.
>
> These are different acts of trust, and a citizen cannot otherwise tell which one they are performing.

Splitting the two dimensions also buys expressiveness the old type lacked: a party whipped on defence and live on everything else, which is what actual parties appear to be.

7 Weight

A voter's weight on an issue is the number of citizens whose resolution reaches them on that issue. So weight is an **output of resolution, not an input**, and it is per-issue: the same person carries different weight on health and on defence.

Two consequences.

Re-convergence. With sub-parties, a citizen may reach Ali directly and also through party A and also through faction X. Ali must not count three times for one citizen, so weight is a count of *citizens reached, deduplicated at the source*, and it is not a local computation.

A circularity to refuse. If a `Weighted` fold counts each member by the weight they carry, and weight flowing *through* the party were counted toward its own members, the fold's input would depend on the fold's output. The rule that prevents it is simple and should be written down rather than discovered: **a member's weight, for the purposes of their own party's fold, counts only delegations that do not pass through that party.**

8 What the algebra refuses to decide

Three choices are political rather than mathematical. The design should present them and not make them.

Cycle policy. Detection is settled; what should happen is not. The vote abstains; or falls back to a $\times$ second choice; or returns to the citizen as a notification and waits; or is divided evenly round the ring. Four theories of what a delegated vote *is*. `Ballot` is deliberately wide enough for all four.

A fold meeting a failure. If one branch cycles or exhausts, does the party abstain wholesale, or fold over the branches that answered? Folding over the survivors is kinder and quietly moves the majority threshold. This is the first thing to settle, because `resolve` cannot be written without it.

The term of a line. A cached fold is stale by construction. Recomputed per issue, per session, or on a member's demand? That is the difference between a whip and a dead hand, and the type carries it in `Term` so that it has to be answered.

9 What to build first

1. The issue tree and `delegateFor` with inheritance. Everything rests on it and it is small.
2. `resolve` as a tree fold, with `visited` and `fuel` in the prompt, returning a `Ballot`. Cycle policy and fold-meets-failure both pluggable, defaulting to abstain, so the arguments can be had against running code.
3. Weights, deduplicated at the source, and a tally for one leaf issue.
4. Only then: voters as separate processes, one per voter, and the chat room as $\boxtimes$ over participants.

10 What is assumed, and what would break it

That a worker's state may be a set, since `visited` and `fuel` ride in the prompt exactly as they do in `mobius`. That question is out to Strathclyde and unanswered.

That the electorate may be unbounded. Every voter serves the *same* container, so

$$\Sigma\,(v : V).\,\mathrm{IssueC} \;\cong\; V \times \mathrm{IssueC},$$

and a sum of a *constant* family is a product with its index set, which does not need the index finite. This is the one place where voting is easier than the firm — there the family genuinely varied with the state, and only a finite one could be summed.

It is worth knowing exactly what would destroy it. Give parties a fibre individuals lack — a manifesto, a whip, a membership roll — and the family stops being constant, the isomorphism fails, and unbounded membership goes with it. That is why `Disclose` is answered by *everybody*, a leaf replying `"leaf"`. Transparency had to be built as a uniform capability or not at all.